

PROJECT REPORT

E-Commerce Product Recommendation System

Data Analytics Internship | Codec Technologies

Submitted By	Rifa Fathima
Internship	Data Analytics — Codec Technologies
Project No.	Project 2 of 2
Core Methods	SVD Matrix Factorization, Content-Based Filtering
Stack	Python, Scikit-surprise, Scikit-learn, Streamlit
Date	5 June 2026

1. Summary

This report presents the complete development of an E-Commerce Product Recommendation System built as part of the Data Analytics Internship at Codec Technologies. The system personalises product discovery by predicting which items a user is most likely to purchase or rate highly, based on their historical interaction data.

Two complementary filtering approaches were implemented: Content-Based Filtering using cosine similarity on product rating profiles, and Collaborative Filtering using Singular Value Decomposition (SVD) — a matrix factorization technique that learns latent user and item representations from the rating matrix. SVD was chosen as the primary model due to its superior performance on sparse datasets and its memory-efficient design. KNN-based approaches were evaluated during development but discarded due to MemoryError failures caused by the extreme sparsity and scale of the Amazon Beauty ratings dataset.

The final SVD model was evaluated using RMSE, MAE, Precision@K, Recall@K, and NDCG@K metrics. An interactive three-tab Streamlit application was built for real-time personalised recommendation delivery.

2. Business Problem Definition

2.1 Problem Statement

E-commerce platforms list thousands to millions of products. Customers have limited time and attention — they will only explore a small fraction of the catalogue in any session. Without personalisation, products that a customer would genuinely love remain hidden, leading to missed purchases, lower session engagement, and reduced retention.

Amazon attributes approximately 35% of its total revenue to its recommendation engine. Netflix estimates its recommendation system saves over one billion dollars annually in reduced churn. For any e-commerce business, a well-designed recommendation system is one of the highest-ROI investments in the data stack.

2.2 Business Objective

Build a system that, given a user's historical rating behaviour, recommends the top N products they are most likely to purchase or rate highly — with a focus on ranking quality (the right products appearing at the top of the list) rather than just prediction accuracy.

2.3 Key Challenges

- Data sparsity — most users rate fewer than 5 products out of thousands; the matrix is over 99% empty
- Cold start — new users and new products have no rating history to learn from
- Scalability — naive dense matrix operations cause MemoryError at this dataset scale
- Evaluation — simple accuracy metrics are insufficient; ranking quality metrics like NDCG are required

2.4 Success Criteria

- SVD RMSE below 1.2 on the held-out test set
- NDCG@10 above 0.35 — the recommended items should be meaningfully ranked
- A Streamlit app that returns personalised recommendations for any user in real time

3. Dataset Description

3.1 Source

The Amazon Product Ratings dataset (Beauty category) was sourced from Kaggle. It contains user ratings for beauty products scraped from Amazon, with four columns: `userId`, `productId`, `rating` (1–5), and `timestamp`.

Statistic	Raw Dataset	Filtered Dataset
Total Ratings	~2,000,000+	~100,000+
Unique Users	~1,200,000+	~10,000+
Unique Products	~250,000+	~5,000+
Sparsity	>99.99%	>99.5%
Filter Threshold	—	Users ≥ 5 ratings, Products ≥ 10 ratings

3.2 Preprocessing — Cold Start Filtering

To reduce sparsity and address the cold start problem, users with fewer than 5 ratings and products with fewer than 10 ratings were removed before model training. This is a standard industry practice that significantly improves model quality by ensuring every entity in the training data has sufficient signal to learn from.

4. Exploratory Data Analysis

4.1 Rating Distribution

- Ratings are strongly left-skewed — the majority of ratings are 4 or 5 stars
- This reflects a selection bias: users tend to rate products they liked enough to purchase and review
- The mean rating across the filtered dataset is approximately 4.1 out of 5

4.2 Long-Tail Distribution

Both users and products follow a power-law (long-tail) distribution: a small fraction of products receive the overwhelming majority of ratings, while most products receive very few. This is a defining characteristic of e-commerce data and is a core reason why memory-based collaborative filtering struggles at scale.

4.3 Sparsity Analysis

The user-item matrix is over 99% sparse — meaning over 99 out of every 100 (user, product) pairs have no rating. Storing this as a dense matrix at the scale of the raw dataset would require multiple gigabytes of RAM and cause `MemoryError` on standard hardware. This was the key motivation for adopting sparse matrix operations (`scipy.sparse.csr_matrix`) and latent factor models (SVD) as the core implementation approach.

4.4 Key EDA Findings

- Dense `pivot_table` approaches are infeasible at this dataset scale — sparse representations are required
- Most users have rated fewer than 10 products — collaborative filtering must handle extreme data scarcity
- Filtering to active users and popular products preserves data quality while reducing sparsity
- Rating bias (towards high scores) means models must be designed to predict relative preference, not absolute rating value

5. Recommendation Methodology

5.1 Approach 1 — Content-Based Filtering

Content-based filtering recommends products similar to those a user has previously rated highly, based on the product's own features. In this implementation, each product is represented by a feature vector of three statistics derived from its rating history: average rating, number of ratings, and standard deviation of ratings.

Cosine similarity is computed between all product feature vectors using sklearn's `cosine_similarity` function. For a given product, the top-N most similar products are returned as recommendations. This approach is particularly useful for the cold start scenario: a new user with only one or two rated products can still receive recommendations without requiring a full rating history.

5.2 Approach 2 — SVD Matrix Factorization (Primary Model)

Singular Value Decomposition (SVD) is a latent factor model that decomposes the user-item rating matrix R into three matrices — a user factor matrix, a diagonal singular value matrix, and an item factor matrix — such that their product approximates R . Each user and each item is represented as a dense vector of latent factors (`n_factors=50`) that capture abstract preferences and characteristics not explicitly present in the data.

SVD was implemented using the Surprise library. The model was trained on 80% of the filtered ratings and evaluated on the held-out 20% test set. After evaluation, the model was retrained on the full filtered dataset to maximise coverage before deployment in the Streamlit application.

5.3 Why SVD over KNN-Based Methods

During development, KNN-based collaborative filtering methods (`KNNWithMeans`, both user-based and item-based) were evaluated. Both approaches failed with `MemoryError` during the similarity matrix computation step — the KNN algorithm in Surprise attempts to build a dense $N \times N$ similarity matrix (where N is the number of users or items), which is infeasible at this dataset scale.

Criterion	KNN-Based (Discarded)	SVD (Adopted)
Memory Usage	$O(N^2)$ dense similarity matrix — <code>MemoryError</code>	$O(N \times K)$ latent factors — scales easily
Scalability	Fails beyond ~10,000 users/items	Handles millions of ratings comfortably
Sparsity Handling	Degrades sharply with sparse data	Designed for sparse matrices
Prediction Quality	Moderate — depends on neighbourhood size	Strong — learns global latent structure
Cold Start	No — requires existing ratings	Partial — handled via content-based fallback
Status in Project	Replaced due to <code>MemoryError</code>	Primary model — deployed in Streamlit app

6. Sparse Matrix Implementation

A key engineering contribution of this project was replacing the naive dense pivot table approach with a scipy sparse matrix pipeline. This resolved the `MemoryError` and enabled the full collaborative filtering workflow to run on standard hardware.

Component	Original (Broken)	Fixed (Sparse)
Matrix Construction	<code>df.pivot_table().fillna(0)</code> — dense array	<code>scipy.sparse.csr_matrix(ratings, (user_idx, product_idx))</code>
Memory	~8 GB for raw dataset	~5 MB — stores only non-zero entries
Similarity Compute	<code>cosine_similarity(dense_df)</code>	<code>cosine_similarity(sparse, dense_output=False)</code>
User Lookup	DataFrame <code>.loc</code> indexing	Integer index maps (<code>user_to_idx</code> , <code>idx_to_user</code>)
Saved Artifact	<code>user_item_matrix.pkl</code> (~8 GB)	<code>sparse_matrix.npz</code> + <code>index_maps.pkl</code> (~5 MB)

7. Model Evaluation

7.1 Evaluation Metrics

Because this is a ranking problem — recommending an ordered list of products — both prediction accuracy metrics and ranking quality metrics were used:

Metric	Formula	Interpretation
RMSE	$\sqrt{\text{mean}((\text{predicted} - \text{actual})^2)}$	Average rating prediction error (lower is better)
MAE	$\text{mean}(\text{predicted} - \text{actual})$	Mean absolute rating error (lower is better)
Precision@K	Relevant items in top-K / K	Fraction of recommended items that are actually relevant
Recall@K	Relevant items in top-K / All relevant items	Fraction of all relevant items captured in top-K
NDCG@K	$\text{DCG@K} / \text{IDCG@K}$ (normalised discounted cumulative gain)	Ranking quality — rewards correct ordering of relevant items

7.2 SVD Performance Results

Model	RMSE	MAE	P@10	R@10	NDCG@10
SVD (n_factors=50)	1.1218	0.8659	0.7794	0.8622	0.8377

The SVD model achieves strong RMSE/MAE scores and meaningful NDCG@10, indicating that the top-ranked recommendations are genuinely relevant to the user. Precision@10 of ~0.38 means approximately 4 out of every 10 recommendations are items the user would rate 4 or above.

7.3 SVD Hyperparameters

Parameter	Value	Description
n_factors	50	Number of latent dimensions — represents abstract user/item preferences
n_epochs	20	Number of SGD training iterations over the training set
lr_all	0.005	Learning rate for all parameters — controls gradient step size
reg_all	0.02	Regularisation term — prevents overfitting on sparse data
random_state	42	Reproducibility seed

8. Hybrid Recommendation Strategy

A hybrid recommender was implemented in the `recommend.py` module that combines SVD predictions with content-based similarity scores. This addresses two limitations of SVD alone:

- Cold start for users with very few ratings — content-based filtering can still generate recommendations
- Diversity — purely collaborative recommendations can be too narrow; content-based introduces variety

The hybrid score is computed as: $\text{hybrid} = 0.7 \times \text{SVD_score} + 0.3 \times \text{content_similarity_score}$. Both scores are min-max normalised before combining. The SVD weight (0.7) reflects its superior ranking quality, while the content weight (0.3) provides cold-start coverage.

9. Streamlit Web Application

9.1 Application Architecture

The Streamlit application (streamlit_app.py) is organised into three tabs, each delivering a different recommendation perspective to the user:

Tab	Method	Description
Tab 1 — User Recommendations	SVD Matrix Factorization	Select a user ID and get their top N personalised product recommendations with predicted ratings
Tab 2 — Product Similarity	Content-Based Filtering	Select a product and discover similar products ranked by cosine similarity with rating statistics
Tab 3 — Dataset Overview	Analytics Dashboard	Live dataset statistics, rating distribution bar chart, and top most-rated products table

9.2 Key Features

- Lazy-loaded model artifacts using `@st.cache_resource` — models load once and are reused across sessions
- Sparse matrix loaded via `scipy.sparse.load_npz` — no memory issues at runtime
- Colour-coded predicted rating table — green for predicted rating ≥ 4.0 , orange for 3.0–4.0
- User history display alongside recommendations — shows what the user has already rated highly
- Dataset overview tab with live KPI metrics and interactive charts

9.3 Running the Application

```
streamlit run app/streamlit_app.py
```

10. Technology Stack

Category	Tool / Library	Purpose
Language	Python 3.13	Core development language
Data Handling	Pandas, NumPy	Data loading, filtering, feature engineering
Sparse Matrices	SciPy (csr_matrix)	Memory-efficient user-item matrix — replaces dense pivot table
Similarity	Scikit-learn	Cosine similarity for content-based filtering
SVD Model	Scikit-surprise (SVD)	Matrix factorization — primary recommendation model
Serialisation	Joblib, SciPy (.npz)	Saving SVD model, index maps, and sparse matrix
Visualisation	Matplotlib, Seaborn	EDA charts and sparsity heatmap
Web App	Streamlit	Three-tab interactive recommendation dashboard
Notebook	Jupyter Notebook	EDA and training workflow

11. Project Structure

```
recommendation_system/  
  data/ ratings.csv ratings_filtered.csv  
  notebooks/ eda.ipynb model_training.ipynb  
  app/ streamlit_app.py recommend.py svd_model.pkl  
      sparse_matrix.npz index_maps.pkl content_sim_matrix.pkl  
      product_features.pkl ratings_clean.csv  
  report/ final_report.pdf  
  requirements.txt README.md
```

12. Conclusion and Business Impact

The E-Commerce Product Recommendation System successfully delivered a full end-to-end recommendation pipeline that operates efficiently at scale. The choice of SVD-based matrix factorization over KNN-based methods was both technically necessary — KNN failed with `MemoryError` on this dataset — and algorithmically superior, as SVD produces better ranking quality by learning global latent structure from the entire rating matrix rather than relying on local neighbourhood similarity.

The combination of content-based filtering (for cold-start coverage) and SVD (for personalised ranking) forms a hybrid system that is representative of production recommendation architectures used in industry. The Streamlit application makes both methods accessible through a clean three-tab interface with no technical knowledge required.

12.1 Business Value

- Increased product discovery — users find relevant products they would not have searched for
- Higher average order value — personalised suggestions surface complementary products
- Improved retention — a platform that understands preferences keeps users returning
- Scalable architecture — sparse matrix and SVD approach scales to millions of ratings

12.2 Future Improvements

- Implement SVD++ — an extension of SVD that incorporates implicit feedback (views, clicks) alongside explicit ratings
- Add A/B testing framework to compare recommendation strategies on click-through rate
- Integrate real-time user feedback loop — update model incrementally as new ratings arrive
- Explore neural collaborative filtering (NCF) using PyTorch for deeper latent representations
- Deploy with FastAPI backend for production-grade REST endpoint serving

References

1. Amazon Product Ratings Dataset — <https://www.kaggle.com/datasets/skillsmuggler/amazon-ratings>
2. Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. IEEE Computer, 42(8), 30-37.
3. Hug, N. (2020). Surprise: A Python library for recommender systems. Journal of Open Source Software, 5(52), 2174.
4. Scikit-learn Documentation — <https://scikit-learn.org/stable/>
5. SciPy Sparse Matrix Documentation — <https://docs.scipy.org/doc/scipy/reference/sparse.html>
6. Streamlit Documentation — <https://docs.streamlit.io/>